THE COMPLETE BACKEND ENGINEERING GUIDE

ASP.NET Core & .NET Backend Mastery

From First Principles to Production-Ready Systems

- MVC Architecture & Web APIs
- Routing, Middleware & Dependency Injection
- REST APIs, JWT Auth & Role-Based Access
- SQL Server + Entity Framework Core
- Clean Architecture & Microservices
- Logging, Exception Handling & API Security
- Docker, CI/CD & Azure Deployment
- Capstone: Production E-Commerce Backend

A hands-on, code-first guide

Every chapter ends with a Practice Lab. Build the capstone project and you are production-ready.

BEFORE YOU BEGIN

Foreword & How To Use This Book

This book was built for one purpose: to take you from writing your first ASP.NET Core endpoint to shipping a production-grade backend with authentication, a real database, clean architecture, and a cloud deployment pipeline — the same shape of system you will be asked to build in a real job.

Who this book is for

- You already write backend code (Node/Express, Django, Spring — doesn't matter which) and want to add C#/.NET to your toolkit.
- You know what an API, a database, and an HTTP request/response cycle are. We will not re-explain those.
- You want **working code you can run**, not just theory slides.

How each chapter is structured

- **Concept first** — the idea explained in plain English, with a diagram or table where it helps.
- **Code second** — a complete, runnable snippet, followed by a line-by-line explanation of the non-obvious parts.
- **Callout boxes** — blue **NOTE** boxes add context, green **PRO TIP** boxes save you debugging time, orange **WATCH OUT** boxes flag the mistakes juniors make in production.
- **Key Takeaways** — a 30-second recap you can revisit before an interview.
- **Practice Lab** — exercises at the end of every chapter. Do not skip these. Reading about middleware and writing middleware are two different skills, and only one of them gets you hired.

> **PRO TIP — The one rule that matters most**
> Type every code sample yourself instead of copy-pasting. You will hit compiler errors, typos, and missing using-statements — and fixing those is exactly the skill that separates someone who has "seen" ASP.NET Core from someone who can actually ship with it.

What you need installed before Chapter 2

Tool	Purpose	Notes
.NET SDK (8.0 LTS)	Compiler, CLI (dotnet), runtime	Run `dotnet --version` to confirm install
Visual Studio 2022 (Community) *or* VS Code + C# Dev Kit	Editor / IDE	VS Code is lighter; Visual Studio has a richer debugger

Tool	Purpose	Notes
SQL Server (Developer/Express) *or* Docker	Relational database for Chapter 3 onward	Docker route is shown so you don't need a Windows-only install
Postman or Thunder Client	Manually call your APIs while building	Thunder Client lives inside VS Code
Docker Desktop	Containerization, Chapter 5	Optional until Chapter 5, install early to avoid a mid-book detour
Git + a GitHub account	Version control, CI/CD in Chapter 5	You already use this daily — nothing new here

Table of Contents

CHAPTER 1

ASP.NET Core Basics

*ASP.NET Core is a free, open-source, cross-platform framework for building APIs and web applications in C#. If you've worked with Express.js, a lot of the mental model transfers directly — middleware pipelines, routing, request/response objects. The biggest difference is that ASP.NET Core leans heavily on **strong typing** and **built-in Dependency Injection**, which is what makes large C# codebases stay maintainable.*

1.1 How an ASP.NET Core app actually boots up

Every ASP.NET Core app starts from a single file: `Program.cs`. Since .NET 6, there's no separate `Startup.cs` anymore — everything lives in one place using what's called the **Minimal Hosting Model**. Think of it as the equivalent of your `app.js` in Express: you create an app, register services, register middleware, then start listening for requests.

●●● Program.cs — the entry point of every ASP.NET Core app C#

```
var builder = WebApplication.CreateBuilder(args);

// 1) REGISTER SERVICES into the DI container (before the app is built)
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// 2) CONFIGURE THE HTTP REQUEST PIPELINE (middleware, in order)
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

// 3) START LISTENING
app.Run();
```

Read that file as two phases. Phase one, everything before `builder.Build()`, is **configuration time** — you're telling the app what services exist ("I will need a database context, a logger, an email service"). Phase two, everything after, is **request time** — you're telling the app what to do with every incoming HTTP request, step by step, in order. This ordering matters enormously and we'll come back to it in Section 1.5.

> **NOTE — builder vs. app**
> `builder` is a construction site — you're assembling services. `app` is the finished building — it's live and serving traffic. You cannot register a new service on `app`; that door closes the moment `Build()` runs.

1.2 MVC Architecture — Model, View, Controller

MVC splits an application into three responsibilities so that no single file has to know everything. For a pure backend/API project (which is what this book focuses on) you will mostly work with Models and Controllers — Views are for server-rendered HTML pages, which matters less when your frontend is a separate React/Next.js app talking to your API over JSON.

Layer	Responsibility	Typical file
Model	Shape of your data + business rules. Plain C# classes.	Models/Product.cs
View	HTML rendering (only relevant for server-rendered MVC apps, not pure APIs)	Views/Product/Index.cshtml
Controller	Receives HTTP requests, talks to services/database, returns a response	Controllers/ProductController.cs

●●● Models/Product.cs — a plain C# Model C#

```
namespace ShopApi.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public decimal Price { get; set; }
        public int StockQuantity { get; set; }
        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    }
}
```

Notice the `{ get; set; }` syntax — these are **properties**, C#'s built-in way of exposing fields with optional logic on read/write. They're the C# equivalent of a plain object field in JS, but type-checked at compile time. `string.Empty` avoids a null reference by default — a category of bug you'll deal with far less once you get comfortable with C#'s nullable reference types.

1.3 Building Web APIs

A **Controller** is a class that groups related endpoints together. It inherits from `ControllerBase` (not `Controller`, which adds View support you don't need for a pure API) and is decorated with attributes that tell ASP.NET Core how to route requests into it.

Controllers/ProductsController.cs — a complete API controller C#

```
using Microsoft.AspNetCore.Mvc;
using ShopApi.Models;

namespace ShopApi.Controllers
{
    [ApiController]                 // enables automatic model validation &
        binding
    [Route("api/[controller]")]     // "[controller]" becomes "products" (class
        name minus "Controller")
    public class ProductsController : ControllerBase
    {
        private static readonly List<Product> _products = new();

        // GET api/products
        [HttpGet]
        public ActionResult<IEnumerable<Product>> GetAll()
        {
            return Ok(_products);
        }

        // GET api/products/5
        [HttpGet("{id:int}")]
        public ActionResult<Product> GetById(int id)
        {
            var product = _products.FirstOrDefault(p => p.Id == id);
            if (product is null) return NotFound();
            return Ok(product);
        }

        // POST api/products
        [HttpPost]
        public ActionResult<Product> Create(Product newProduct)
        {
            newProduct.Id = _products.Count + 1;
            _products.Add(newProduct);
            return CreatedAtAction(nameof(GetById), new { id = newProduct.Id },
                newProduct);
        }
    }
}
```

Walking through the non-obvious parts: `[ApiController]` switches on several conveniences automatically — it returns `400 Bad Request` for invalid models without you writing that check, and it infers where parameters come from (route, query string, or body). `ActionResult<T>` lets a single method return either the actual data (`T`) or an HTTP status like `NotFound()` — this is the idiomatic return type for API endpoints. `CreatedAtAction` is the “correct” way to respond to a POST: it returns `201 Created` plus a `Location` header pointing at the new resource, which is what REST purists (and API design linters) expect.

> **WATCH OUT — In-memory lists are a demo trick, not a database**
> The `_products` list above resets every time the app restarts and is not thread-safe under concurrent requests. It's here so you can see routing and controllers in isolation before Chapter 2 introduces a real database via Entity Framework Core. Never ship this pattern.

1.4 Routing — conventional vs. attribute routing

Routing is the process of matching an incoming URL + HTTP verb to a specific method in your code. ASP.NET Core supports two styles.

Style	How it works	When to use it
Conventional routing	One central pattern defined once (e.g. `{controller}/{action}/{id?}`) that applies to every controller	Classic server-rendered MVC websites
Attribute routing	Routes declared directly above each controller/action with `[Route]`, `[HttpGet]` etc.	Web APIs — the industry standard today, used throughout this book

Attribute routing in detail C#

```
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    // Route becomes: GET api/orders
    [HttpGet]
    public IActionResult GetAll() => Ok();

    // Route becomes: GET api/orders/42
    [HttpGet("{id:int}")]
    public IActionResult GetOne(int id) => Ok();

    // Route becomes: GET api/orders/42/items
    [HttpGet("{id:int}/items")]
    public IActionResult GetOrderItems(int id) => Ok();

    // Route becomes: GET api/orders/search?status=paid
    [HttpGet("search")]
    public IActionResult Search([FromQuery] string status) => Ok();
}
```

> **PRO TIP — Route constraints save you an if-statement**
> `{id:int}` is a **route constraint** — it tells the router to only match this route if the segment is a valid integer, and to skip straight to a 404 otherwise. Other built-ins include `:guid`, `:alpha`, `:min(1)`, and `:regex(...)`.

KEY TAKEAWAYS

- Program.cs has two phases: register services first, configure the middleware pipeline second.
- MVC keeps data (Model), presentation (View), and request-handling (Controller) separate; APIs mostly use Model + Controller.
- [ApiController] + ActionResult<T> is the standard shape of a modern API controller method.
- Attribute routing ([Route], [HttpGet], etc.) is what you'll use for virtually all API work.

1.5 Middleware — the request pipeline

Every HTTP request that hits your app travels through a chain of components called **middleware**, one after another, like an assembly line (or Express's `app.use()` chain, if that's familiar). Each piece of middleware can inspect the request, modify it, short-circuit it (send a response immediately), or pass it along to the next piece with `await next()`.

The pipeline, visualized as code C#

```
app.UseHttpsRedirection();   // 1. redirect http -> https
app.UseRouting();            // 2. figure out which endpoint matches
app.UseAuthentication();     // 3. who is the caller? (reads the JWT, sets User)
app.UseAuthorization();      // 4. is the caller allowed to hit this endpoint?
app.MapControllers();        // 5. run the matched controller action
```

Order is not cosmetic — it's the single most common source of “why is my auth not working” bugs. If `UseAuthorization()` runs before `UseAuthentication()`, every request looks unauthenticated because the pipeline never had a chance to read the token yet.

Writing your own custom middleware

You'll frequently need custom middleware — logging every request's duration, adding a correlation ID, or catching unhandled exceptions globally (we'll build a real exception-handling middleware in Chapter 3). Here's the minimal shape:

●●● Middleware/RequestTimingMiddleware.cs — custom middleware, class-based C#

```
namespace ShopApi.Middleware
{
    public class RequestTimingMiddleware
    {
        private readonly RequestDelegate _next;
        private readonly ILogger<RequestTimingMiddleware> _logger;

        // ASP.NET Core's DI container supplies both parameters automatically
        public RequestTimingMiddleware(RequestDelegate next,
            ILogger<RequestTimingMiddleware> logger)
        {
            _next = next;
            _logger = logger;
        }

        public async Task InvokeAsync(HttpContext context)
        {
            var stopwatch = System.Diagnostics.Stopwatch.StartNew();

            await _next(context);   // hand off to the next middleware in line

            stopwatch.Stop();
            _logger.LogInformation(
                "{Method} {Path} responded {StatusCode} in {Elapsed}ms",
                context.Request.Method, context.Request.Path,
                context.Response.StatusCode, stopwatch.ElapsedMilliseconds);
        }
    }
}

// Extension method so it reads nicely in Program.cs
public static class RequestTimingMiddlewareExtensions
{
    public static IApplicationBuilder UseRequestTiming(this IApplicationBuilder
        builder)
        => builder.UseMiddleware<RequestTimingMiddleware>();
}

// Program.cs
app.UseRequestTiming();   // add this near the top of the pipeline
```

The pattern to memorize: a constructor that captures `RequestDelegate next` (a pointer to "whatever comes after me"), and an `InvokeAsync` method that does work *before* calling `await _next(context)`, work *after* it, or both. Skipping the call to `_next` entirely is how you short-circuit a request — useful for things like a maintenance-mode middleware that returns 503 for every request.

PRO TIP — Prefer app.Use(...) for quick, one-off middleware

For something short you don't need a whole class for, you can inline it directly in `Program.cs`: `app.Use(async (context, next) => { /* before */ await next(); /* after */ });` Reach for a full middleware class once the logic grows past a few lines or needs injected services beyond the logger.

1.6 Dependency Injection

Dependency Injection (DI) is baked into ASP.NET Core at the framework level — you don't add a third-party library for it, unlike in many Node/Express setups. The core idea: instead of a class creating its own dependencies with `new SomeService()`, it *declares what it needs* in its constructor, and the framework hands it a ready-made instance. This makes classes far easier to unit test, because in a test you can hand them a fake/mock implementation instead.

Without DI vs. with DI C#

```
// WITHOUT DI — tightly coupled, hard to test
public class OrderService
{
    private readonly SqlEmailSender _emailSender = new SqlEmailSender(); //
        hard-wired
    public void PlaceOrder(Order order) =>
        _emailSender.Send(order.CustomerEmail, "Order placed!");
}

// WITH DI — depends on an abstraction, not a concrete class
public interface IEmailSender
{
    void Send(string to, string message);
}

public class OrderService
{
    private readonly IEmailSender _emailSender;

    public OrderService(IEmailSender emailSender)   // <-- injected, not
        "new"-ed
    {
        _emailSender = emailSender;
    }

    public void PlaceOrder(Order order) =>
        _emailSender.Send(order.CustomerEmail, "Order placed!");
}
```

Registering services in Program.cs C#

```
builder.Services.AddScoped<IEmailSender, SendGridEmailSender>();
builder.Services.AddScoped<OrderService>();

// Now anywhere OrderService is requested (e.g. injected into a controller),
// ASP.NET Core builds it automatically, resolving IEmailSender for you:

[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _orderService;
    public OrdersController(OrderService orderService) => _orderService =
        orderService;

    [HttpPost]
    public IActionResult Place(Order order)
    {
        _orderService.PlaceOrder(order);
        return Ok();
    }
}
```

The three service lifetimes

Lifetime	A new instance is created...	Typical use case
Transient	every single time it's requested	Lightweight, stateless services (e.g. a formatter/validator)
Scoped	once per HTTP request, reused within that request	Database contexts (EF Core's AppDbContext) — the default for most business services
Singleton	once for the entire lifetime of the app	Configuration objects, in-memory caches, connection pools

WATCH OUT — Never inject a Scoped service into a Singleton

This is the #1 DI bug in real ASP.NET Core apps: a Singleton service captures a Scoped service (like your `AppDbContext`) at startup and keeps reusing that same instance forever, across every request — leading to data from one user's request leaking into another's, or a crash. If you truly need database access inside a Singleton, inject `IServiceScopeFactory` and create a fresh scope manually.

PRACTICE LAB — CHAPTER 1 EXERCISES

1. Scaffold a new ASP.NET Core Web API project (`dotnet new webapi -n TaskApi`) and run it. Confirm Swagger UI loads in the browser.

Hint / Goal: You should see the default WeatherForecast endpoint documented automatically.

2. Build a `TasksController` with an in-memory list, exposing GET (all), GET by id, and POST, following the ProductsController pattern above.

Hint / Goal: Use ActionResult<T>, [ApiController], and CreatedAtAction for the POST response.

3. Write a custom middleware that adds a response header `X-Correlation-Id` with a new GUID on every request.

Hint / Goal: Generate the GUID with Guid.NewGuid().ToString(), and set it via context.Response.Headers before calling _next.

4. Deliberately register the middleware from the previous exercise *after* app.MapControllers() and observe that the header never gets set. Explain why in one sentence.

Hint / Goal: This proves middleware order matters — anything after MapControllers() only runs for requests that don't match a route.

5. Create an ILogger, IEmailSender interface + a FakeEmailSender that just writes to the console, register it as Scoped, and inject it into a controller.

Hint / Goal: Confirms you can wire up your own service through the DI container end-to-end.

CHAPTER 2

Backend Development & Database

This is where the book turns into real backend engineering: a full REST API with proper CRUD, JWT-based authentication, role-based authorization, and a real SQL Server database wired up through Entity Framework Core. By the end of this chapter you'll be able to build the same kind of API that powers a production SaaS product.

2.1 REST API design and full CRUD

CRUD stands for Create, Read, Update, Delete — the four operations almost every resource in a backend needs. REST maps these onto HTTP verbs in a predictable way, and following that convention is what makes an API feel professional instead of ad-hoc.

Operation	HTTP Verb	Route example	Success status
Create	POST	POST /api/products	201 Created
Read (all)	GET	GET /api/products	200 OK
Read (one)	GET	GET /api/products/5	200 OK
Update (full)	PUT	PUT /api/products/5	204 No Content
Update (partial)	PATCH	PATCH /api/products/5	204 No Content
Delete	DELETE	DELETE /api/products/5	204 No Content

●●● Controllers/ProductsController.cs — full CRUD against a real DbContext C#

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly AppDbContext _context;
    public ProductsController(AppDbContext context) => _context = context;

    [HttpGet]
    public async Task<ActionResult<IEnumerable<Product>>> GetAll()
        => Ok(await _context.Products.ToListAsync());

    [HttpGet("{id:int}")]
    public async Task<ActionResult<Product>> GetById(int id)
    {
        var product = await _context.Products.FindAsync(id);
        return product is null ? NotFound() : Ok(product);
    }

    [HttpPost]
    public async Task<ActionResult<Product>> Create(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();
        return CreatedAtAction(nameof(GetById), new { id = product.Id },
            product);
    }

    [HttpPut("{id:int}")]
    public async Task<IActionResult> Update(int id, Product updated)
    {
        if (id != updated.Id) return BadRequest("Route id and body id must
            match.");

        var exists = await _context.Products.AnyAsync(p => p.Id == id);
        if (!exists) return NotFound();

        _context.Entry(updated).State = EntityState.Modified;
        await _context.SaveChangesAsync();
        return NoContent();
    }

    [HttpDelete("{id:int}")]
    public async Task<IActionResult> Delete(int id)
    {
        var product = await _context.Products.FindAsync(id);
        if (product is null) return NotFound();

        _context.Products.Remove(product);
        await _context.SaveChangesAsync();
        return NoContent();
    }
}
```

Every method here is `async Task<...>` and uses `await` on database calls. This matters at scale: a synchronous call blocks the thread handling that request until the database responds, while an async call frees that thread to serve other requests in the meantime. Under real traffic, this is the difference between

a server that handles hundreds of concurrent users comfortably and one that falls over.

> **PRO TIP — Use DTOs instead of exposing your database entities directly**
> The controller above returns the `Product` entity straight from the database, which is fine for a learning example but risky in production — you might accidentally leak an internal field (a cost price, an internal flag) to the client. The production pattern is a separate **DTO (Data Transfer Object)** class that only contains the fields you want to expose, with a small mapping step between entity and DTO. We use this pattern in the Chapter 5 capstone.

2.2 Authentication with JWT

Authentication answers "who is making this request?" A **JWT (JSON Web Token)** is a signed, self-contained string that encodes a user's identity and claims. The server issues it once at login; after that, the client sends it back on every request (usually in the `Authorization: Bearer <token>` header) and the server verifies its signature instead of hitting the database on every single request to check who's logged in.

●●● A JWT has three parts, separated by dots C#

```
HEADER.PAYLOAD.SIGNATURE

// Decoded, the payload of a real token looks like this:
{
  "sub": "14",                 // the user's id
  "email": "sam@shop.com",
  "role": "Admin",
  "exp": 1735689600            // expiry, as a unix timestamp
}
```

Program.cs — wiring up JWT authentication C#

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var jwtKey = builder.Configuration["Jwt:Key"]!;

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidateAudience = true,
            ValidAudience = builder.Configuration["Jwt:Audience"],
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new
                SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey)),
            ValidateLifetime = true,
            ClockSkew = TimeSpan.Zero
        };
    });

builder.Services.AddAuthorization();

// ... after app.Build():
app.UseAuthentication();   // MUST come before UseAuthorization
app.UseAuthorization();
```

Services/TokenService.cs — issuing a token at login C#

```
public class TokenService
{
    private readonly IConfiguration _config;
    public TokenService(IConfiguration config) => _config = config;

    public string CreateToken(User user)
    {
        var claims = new List<Claim>
        {
            new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
            new Claim(JwtRegisteredClaimNames.Email, user.Email),
            new Claim(ClaimTypes.Role, user.Role)   // "Admin" or "Customer"
        };

        var key = new
            SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _config["Jwt:Issuer"],
            audience: _config["Jwt:Audience"],
            claims: claims,
            expires: DateTime.UtcNow.AddHours(2),
            signingCredentials: creds);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

●●● Controllers/AuthController.cs — the login endpoint C#

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AppDbContext _context;
    private readonly TokenService _tokenService;

    public AuthController(AppDbContext context, TokenService tokenService)
    {
        _context = context;
        _tokenService = tokenService;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login(LoginRequest request)
    {
        var user = await _context.Users
            .FirstOrDefaultAsync(u => u.Email == request.Email);

        if (user is null || !BCrypt.Net.BCrypt.Verify(request.Password,
            user.PasswordHash))
            return Unauthorized("Invalid email or password.");

        var token = _tokenService.CreateToken(user);
        return Ok(new { token });
    }
}
```

WATCH OUT — Never store plain-text passwords

Always hash passwords with a slow, purpose-built algorithm like **BCrypt** (`dotnet add package BCrypt.Net-Next`) before saving them, and compare with `BCrypt.Verify()` at login. Fast general-purpose hashes like MD5 or SHA-256 are unsafe for passwords — they're specifically designed to be fast, which makes brute-forcing them cheap.

2.3 Authorization — role-based and policy-based access

Authorization answers a different question from authentication: not “who are you” but “are you allowed to do this.” Once `UseAuthentication()` has verified the JWT and populated `HttpContext.User` with claims, authorization attributes decide who gets through.

●●● Role-based authorization — the simple case C#

```
[Authorize]                          // any authenticated user
[HttpGet("my-orders")]
public IActionResult GetMyOrders() => Ok();

[Authorize(Roles = "Admin")]         // only users with the "Admin" role claim
[HttpDelete("{id:int}")]
public IActionResult DeleteProduct(int id) => Ok();

[Authorize(Roles = "Admin,Manager")] // either role passes
[HttpPost("bulk-import")]
public IActionResult BulkImport() => Ok();
```

●●● Policy-based authorization — for rules more complex than a role name C#

```
// Program.cs
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("MinimumAge18", policy =>
        policy.RequireAssertion(ctx =>
            ctx.User.HasClaim(c => c.Type == "Age") &&
            int.Parse(ctx.User.FindFirst("Age")!.Value) >= 18));

    options.AddPolicy("CanManageInventory", policy =>
        policy.RequireRole("Admin", "InventoryManager"));
});

// Usage on a controller action:
[Authorize(Policy = "CanManageInventory")]
[HttpPut("{id:int}/stock")]
public IActionResult UpdateStock(int id, int quantity) => Ok();
```

NOTE — Roles vs. Policies — which should you reach for?

Use `Roles` when the rule is really just “which role can access this”. Reach for a **Policy** the moment the rule needs more than a role check — combining multiple claims, checking resource ownership (“can this user edit *this specific* order”), or any custom logic. Policies scale; scattering role-checking if-statements through controllers does not.

2.4 SQL Server integration

SQL Server is accessed through a **connection string** — a single configuration value describing where the database lives and how to authenticate to it. This lives in `appsettings.json` (and, for real secrets, in environment variables or a secrets manager — never committed to source control).

●●● appsettings.json C#

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost,1433;Database=ShopDb;User
        Id=sa;Password=YourPassword123!;TrustServerCertificate=True;"
  },
  "Jwt": {
    "Key": "a-long-random-secret-key-min-32-chars",
    "Issuer": "ShopApi",
    "Audience": "ShopApiClient"
  }
}
```

●●● Running SQL Server locally via Docker (no Windows-only install needed) bash

```
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=YourPassword123!" \
  -p 1433:1433 --name sql-dev -d mcr.microsoft.com/mssql/server:2022-latest
```

●●● Registering the DbContext in Program.cs C#

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConne
        ction")));
```

2.5 Entity Framework Core

Entity Framework Core (EF Core) is an **ORM** — Object-Relational Mapper. It lets you work with your database as C# objects and LINQ queries instead of hand-written SQL strings. Think of it as the C# equivalent of Prisma or Sequelize.

The DbContext — your database, represented as a class

Data/AppDbContext.cs C#

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        { }

    public DbSet<Product> Products => Set<Product>();
    public DbSet<User> Users => Set<User>();
    public DbSet<Order> Orders => Set<Order>();
    public DbSet<OrderItem> OrderItems => Set<OrderItem>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // fine-grained configuration lives here (see Relationships below)
        modelBuilder.Entity<Product>()
            .Property(p => p.Price)
            .HasColumnType("decimal(18,2)");
    }
}
```

Migrations — versioning your database schema like Git versions your code

The migration workflow bash

```
# Install the EF Core CLI tool once per machine
dotnet tool install --global dotnet-ef

# Whenever you change a model class, generate a migration:
dotnet ef migrations add AddProductsTable

# Apply pending migrations to the actual database:
dotnet ef database update

# See what would change without applying it:
dotnet ef migrations script
```

A migration is a C# file EF Core generates describing exactly what changed — new table, new column, new index — along with an `Up()` method (apply the change) and a `Down()` method (undo it). Commit migration files to source control just like any other code; they're how your team keeps every environment's database schema in sync.

Relationships

One-to-many: a Product has many OrderItems C#

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }

    public List<OrderItem> OrderItems { get; set; } = new();  // navigation
        property
}

public class OrderItem
{
    public int Id { get; set; }
    public int Quantity { get; set; }

    public int ProductId { get; set; }        // foreign key
    public Product Product { get; set; } = null!;  // navigation property back
        to Product

    public int OrderId { get; set; }
    public Order Order { get; set; } = null!;
}
```

Loading related data — eager loading with Include() C#

```
var orders = await _context.Orders
    .Include(o => o.OrderItems)
        .ThenInclude(oi => oi.Product)
    .Where(o => o.UserId == currentUserId)
    .ToListAsync();
```

WATCH OUT — The N+1 query trap

If you forget `.Include()` and instead access `order.OrderItems` inside a loop, EF Core (with lazy loading enabled) fires a separate database query for every single order — 1 query for the orders, plus N more for their items. On a page listing 200 orders, that's 201 round-trips instead of 1. This is the single most common EF Core performance bug in production apps. Always `Include()` what you know you'll need.

Repository Pattern

The Repository Pattern wraps EF Core's `DbContext` behind an interface, so your controllers and services depend on an abstraction rather than EF Core directly. This pays off in two ways: it's easier to unit-test (mock the interface instead of a real database), and it centralizes query logic instead of scattering LINQ across every controller.

●●● Repositories/IProductRepository.cs + implementation C#

```
public interface IProductRepository
{
    Task<IEnumerable<Product>> GetAllAsync();
    Task<Product?> GetByIdAsync(int id);
    Task AddAsync(Product product);
    Task<bool> DeleteAsync(int id);
}

public class ProductRepository : IProductRepository
{
    private readonly AppDbContext _context;
    public ProductRepository(AppDbContext context) => _context = context;

    public async Task<IEnumerable<Product>> GetAllAsync()
        => await _context.Products.AsNoTracking().ToListAsync();

    public async Task<Product?> GetByIdAsync(int id)
        => await _context.Products.FindAsync(id);

    public async Task AddAsync(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();
    }

    public async Task<bool> DeleteAsync(int id)
    {
        var product = await _context.Products.FindAsync(id);
        if (product is null) return false;
        _context.Products.Remove(product);
        await _context.SaveChangesAsync();
        return true;
    }
}

// Program.cs
builder.Services.AddScoped<IProductRepository, ProductRepository>();
```

PRO TIP — AsNoTracking() for read-only queries

When you're only reading data and won't call `SaveChanges()` on it, add `.AsNoTracking()`. It tells EF Core not to bother tracking the entities for changes, which is a meaningful performance win on read-heavy endpoints like a product listing page.

2.6 Code-First vs. Database-First

Approach	How it works	Best for
Code-First	You write C# model classes; EF Core generates migrations that create/update the database schema to match.	New projects, teams that want schema history tracked in Git — the default recommendation, and what this book uses.
Database-First	A database already exists; you run `Scaffold-DbContext` to reverse-engineer C# model classes and a DbContext from it.	Legacy databases you don't control, or integrating with a DB owned by another team/DBA.

Database-First: scaffolding models from an existing SQL Server database bash

```
dotnet ef dbcontext scaffold
    "Server=localhost;Database=LegacyDb;Trusted_Connection=True;" \
  Microsoft.EntityFrameworkCore.SqlServer -o Models --context LegacyDbContext
```

KEY TAKEAWAYS

- REST CRUD maps cleanly onto HTTP verbs: POST=Create, GET=Read, PUT/PATCH=Update, DELETE=Delete.
- A JWT is issued once at login and verified (not looked up) on every subsequent request — that's what makes it stateless and fast.
- [Authorize(Roles="...")] handles simple role checks; custom Policies handle everything more complex.
- EF Core Migrations are how schema changes travel with your code through Git, just like the code itself.
- Include()/ThenInclude() prevent the N+1 query problem — the most common EF Core performance mistake.
- The Repository Pattern isolates EF Core behind an interface, making services testable and queries centralized.

PRACTICE LAB — CHAPTER 2 EXERCISES

1. Add SQL Server + EF Core to the TaskApi from Chapter 1: create a TasksDbContext, run your first migration, and confirm the Tasks table appears in the database.

Hint / Goal: Use dotnet ef migrations add InitialCreate then dotnet ef database update.

2. Add a User entity with Email and PasswordHash, hash a password with BCrypt on registration, and build a /api/auth/login endpoint that returns a JWT.

Hint / Goal: Reuse the TokenService pattern shown above.

3. Add an [Authorize(Roles="Admin")] endpoint that only an Admin-role JWT can call successfully; verify with Postman that a Customer-role token gets a 403.

Hint / Goal: Set the role claim at token creation time and test both cases.

4. Model a one-to-many relationship (e.g. a Category has many Tasks), add a migration for it, and write a query using Include() to fetch a category with its tasks.

Hint / Goal: Confirm in SQL Server Management Studio (or Azure Data Studio) that a foreign key column was created.

5. Refactor your TasksController to use a Repository Pattern (ITaskRepository / TaskRepository) instead of calling the DbContext directly.

Hint / Goal: The controller should no longer have a direct reference to AppDbContext afterward.

CHAPTER 3

Advanced .NET Concepts

Everything so far makes an API that works. This chapter makes it an API that survives contact with production — organized so a team can maintain it, resilient when things go wrong, and hardened against the mistakes that show up in security audits.

3.1 Clean Architecture

Clean Architecture organizes a codebase into concentric layers, where dependencies only ever point **inward**. The innermost layer (your core business logic) knows nothing about the database, the web framework, or any external service — those are all outer, replaceable details. This is the single biggest structural upgrade you can make once a project's controllers start becoming 500-line files mixing HTTP handling, business rules, and raw SQL together.

Layer	Contains	Depends on
Domain	Entities, enums, core business rules — pure C#, zero framework references	Nothing
Application	Use cases / services, interfaces (IProductRepository, IEmailSender), DTOs	Domain only
Infrastructure	EF Core DbContext, repository implementations, external API clients	Application (implements its interfaces)
API / Presentation	Controllers, middleware, Program.cs	Application (calls its use cases)

●●● Typical folder / project structure — text

```
ShopApi.sln
 |-- ShopApi.Domain/
 |     |-- Entities/Product.cs
 |     |-- Entities/Order.cs
 |-- ShopApi.Application/
 |     |-- Interfaces/IProductRepository.cs
 |     |-- Services/ProductService.cs
 |     |-- DTOs/ProductDto.cs
 |-- ShopApi.Infrastructure/
 |     |-- Data/AppDbContext.cs
 |     |-- Repositories/ProductRepository.cs
 |-- ShopApi.Api/
       |-- Controllers/ProductsController.cs
       |-- Program.cs
```

Why this pays off: the `Domain` and `Application` layers can be fully unit-tested without spinning up a database, because they only depend on interfaces, not concrete EF Core classes. And when a future requirement says “swap SQL Server for PostgreSQL” or “add a caching layer”, that change is contained entirely inside `Infrastructure` — your business rules never have to change.

> **NOTE — Is this overkill for a small project?**
> Honestly, sometimes. For a weekend project or a tiny internal tool, three flat folders (Controllers, Models, Services) is fine. Clean Architecture earns its complexity once a codebase has multiple developers, will live for years, or has business logic complex enough that you genuinely need to unit-test it in isolation from the database. The **capstone project** in Chapter 5 uses a simplified version of this layering so you can feel the difference firsthand.

3.2 Microservices basics

A microservices architecture splits an application into multiple small, independently deployable services, each owning its own data and communicating over the network (usually HTTP/REST, gRPC, or a message queue) instead of in-process method calls. It's the opposite end of the spectrum from the single-project “monolith” you've built everywhere else in this book.

	Monolith	Microservices
Deployment	One unit, deployed together	Each service deployed independently
Database	Usually one shared database	Each service typically owns its own database
Team scaling	Gets harder to coordinate as the team grows past ~1-2 squads	Different teams can own different services
Operational cost	Low — one app to monitor, log, deploy	High — needs service discovery, distributed tracing, network resilience
Best for	Most startups, small-to-mid teams, MVPs	Large orgs with clear domain boundaries and dedicated platform/DevOps capacity

> **WATCH OUT — Don't start with microservices**
> This is the most common architecture mistake in the industry right now, not just for beginners — it's a well-known trap even experienced teams fall into. Microservices trade one set of problems (a large codebase) for a much harder set (network latency, partial failures, distributed transactions, eventual consistency, and a real DevOps burden). Almost every successful product, including most you use daily, started as a monolith. Build a well-structured monolith with Clean Architecture first; split out a service later only when you have a concrete, measured reason to.

A minimal taste of inter-service communication

●●● **Calling another microservice with HttpClient (registered via DI)** C#

```
// Program.cs
builder.Services.AddHttpClient("InventoryService", client =>
{
    client.BaseAddress = new
        Uri(builder.Configuration["Services:InventoryUrl"]!);
    client.Timeout = TimeSpan.FromSeconds(5);
});

// Somewhere that needs to call it:
public class OrderService
{
    private readonly HttpClient _inventoryClient;
    public OrderService(IHttpClientFactory factory)
        => _inventoryClient = factory.CreateClient("InventoryService");

    public async Task<bool> IsInStockAsync(int productId)
    {
        var response = await
            _inventoryClient.GetAsync($"/api/stock/{productId}");
        if (!response.IsSuccessStatusCode) return false;
        var stock = await response.Content.ReadFromJsonAsync<StockDto>();
        return stock is not null && stock.Quantity > 0;
    }
}
```

3.3 Logging & Exception Handling

In production you cannot attach a debugger to a live server. Logs are how you find out what happened. ASP.NET Core has structured logging built in via `ILogger<T>`, and pairs well with providers like **Serilog** for writing to files, the console, or a log aggregation service (Seq, Application Insights, ELK).

Structured logging with ILogger<T> C#

```
public class OrdersController : ControllerBase
{
    private readonly ILogger<OrdersController> _logger;
    public OrdersController(ILogger<OrdersController> logger) => _logger =
        logger;

    [HttpPost]
    public async Task<IActionResult> Place(Order order)
    {
        _logger.LogInformation("Placing order for user {UserId} with
            {ItemCount} items",
            order.UserId, order.Items.Count);
        try
        {
            // ... place order ...
            return Ok();
        }
        catch (InsufficientStockException ex)
        {
            _logger.LogWarning(ex, "Order failed due to insufficient stock for
                user {UserId}", order.UserId);
            return Conflict(ex.Message);
        }
    }
}
```

PRO TIP — Log the placeholders, not a concatenated string

Write `_logger.LogInformation("User {UserId} logged in", userId)`, not `_logger.LogInformation($"User {userId} logged in")`. The first form keeps `UserId` as a separate, queryable field in structured log storage — you can filter “show me every log line for UserId=14” later. The second just bakes it into a flat string that's much harder to search.

Global exception handling — one place to catch everything

Middleware/ExceptionHandlingMiddleware.cs C#

```
public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionHandlingMiddleware> _logger;

    public ExceptionHandlingMiddleware(RequestDelegate next,
        ILogger<ExceptionHandlingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Unhandled exception on {Path}",
                context.Request.Path);

            context.Response.ContentType = "application/json";
            context.Response.StatusCode = ex switch
            {
                NotFoundException => StatusCodes.Status404NotFound,
                ValidationException => StatusCodes.Status400BadRequest,
                UnauthorizedAccessException =>
                    StatusCodes.Status401Unauthorized,
                _ => StatusCodes.Status500InternalServerError
            };

            var payload = new { error = ex.Message, traceId =
                context.TraceIdentifier };
            await context.Response.WriteAsJsonAsync(payload);
        }
    }
}

// Program.cs — register it FIRST, so it wraps everything after it
app.UseMiddleware<ExceptionHandlingMiddleware>();
```

This is exactly the same custom-middleware pattern from Section 1.5, applied to a real production concern: one place that converts *any* unhandled exception into a consistent JSON error shape instead of leaking a raw stack trace to the client (a real security issue) or crashing the whole process.

3.4 API Security Best Practices

Practice	Why it matters
Always validate input server-side ([Required], [StringLength], FluentValidation)	Client-side validation can always be bypassed by calling the API directly
Use HTTPS everywhere; call app.UseHttpsRedirection()	Prevents tokens and data from being readable on the network
Never return stack traces or exception details to the client in production	Stack traces reveal internal file paths, library versions, and logic — a gift to attackers
Rate-limit sensitive endpoints (login, password reset)	Stops brute-force and credential-stuffing attacks
Set a short JWT expiry + use refresh tokens for long sessions	Limits the damage window if a token is ever leaked
Store secrets (connection strings, JWT keys) outside source control	Use user-secrets locally, environment variables or Azure Key Vault in production
Enable CORS with an explicit allow-list, never AllowAnyOrigin() in production	Prevents malicious sites from making authenticated requests using a user's browser session
Parameterize all database queries (EF Core does this by default)	Prevents SQL injection — never string-concatenate raw SQL

●●● Rate limiting a login endpoint (built into .NET 7+) C#

```
// Program.cs
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("LoginPolicy", opt =>
    {
        opt.PermitLimit = 5;
        opt.Window = TimeSpan.FromMinutes(1);
        opt.QueueLimit = 0;
    });
});
// after app.Build():
app.UseRateLimiter();

// AuthController.cs
[EnableRateLimiting("LoginPolicy")]
[HttpPost("login")]
public async Task<IActionResult> Login(LoginRequest request) { /* ... */ }
```

KEY TAKEAWAYS

- Clean Architecture keeps business logic independent of the database and web framework — dependencies point inward, toward Domain.
- Default to a well-structured monolith. Reach for microservices only when you have a concrete, measured reason to split.
- Log with structured placeholders ({UserId}), not string interpolation, so logs stay queryable.
- A single exception-handling middleware, registered first in the pipeline, gives you one consistent error response shape everywhere.
- Security is a checklist, not a feature: input validation, HTTPS, no leaked stack traces, rate limiting, short-lived JWTs, and parameterized queries.

PRACTICE LAB — CHAPTER 3 EXERCISES

1. Reorganize the TaskApi from Chapters 1-2 into Domain / Application / Infrastructure / Api projects following the Clean Architecture layout shown above.

Hint / Goal: Use dotnet new classlib for each layer and add project references pointing inward only.

2. Build the ExceptionHandlingMiddleware shown above and throw a custom NotFoundException from a controller on purpose to confirm it returns a clean 404 JSON payload.

Hint / Goal: Define NotFoundException as a simple class inheriting from Exception.

3. Add Serilog to your project, configured to write structured logs to both the console and a rolling file.

Hint / Goal: dotnet add package Serilog.AspNetCore, then builder.Host.UseSerilog(...).

4. Add rate limiting to your login endpoint capping it at 5 attempts per minute per client, and verify with Postman that the 6th rapid attempt returns 429.

Hint / Goal: Use the AddFixedWindowLimiter pattern shown above.

5. Write a one-page security checklist review of your own TaskApi against the 8-item table above — mark each item Done / Not Done / N-A.

Hint / Goal: This is the exact kind of review a senior engineer does before a project ships.

CHAPTER 4

Deployment & Cloud

Code that only runs on your laptop isn't a product. This chapter covers the minimum every backend developer needs to ship: packaging your app into a container, automating the build/test/deploy cycle, and getting it running on Azure. None of this requires you to become a DevOps specialist — just fluent enough to deploy your own work with confidence.

4.1 Docker basics and containerization

A container packages your app together with everything it needs to run — the .NET runtime, your compiled code, configuration — into a single, portable unit. “It works on my machine” stops being a problem because the container *is* the machine, wherever it runs.

●●● Dockerfile — multi-stage build for an ASP.NET Core API — dockerfile

```
# Stage 1: build the app using the full SDK image
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["ShopApi.csproj", "./"]
RUN dotnet restore "ShopApi.csproj"
COPY . .
RUN dotnet publish "ShopApi.csproj" -c Release -o /app/publish

# Stage 2: copy the published output into a lightweight runtime-only image
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "ShopApi.dll"]
```

This is called a **multi-stage build** and it's the standard pattern for .NET. Stage 1 uses the heavy SDK image (which includes the compiler) to build and publish your app. Stage 2 starts fresh from a much smaller runtime-only image and copies in just the compiled output. The result is a final image with no compiler, no source code, and no build tools — smaller, faster to pull, and a smaller attack surface in production.

Building and running the container bash

```
docker build -t shopapi:latest .
docker run -d -p 8080:8080 --name shopapi-container \
  -e
      ConnectionStrings__DefaultConnection="Server=host.docker.internal,1433;Dat
      abase=ShopDb;User
      Id=sa;Password=YourPassword123!;TrustServerCertificate=True;" \
  shopapi:latest
```

NOTE — Double underscore for nested config in environment variables

ASP.NET Core's configuration system maps `ConnectionStrings__DefaultConnection` (double underscore) in an environment variable to the nested JSON path `ConnectionStrings:DefaultConnection` in appsettings.json. This is how you override configuration per-environment without editing files.

docker-compose — running the API and its database together

docker-compose.yml yaml

```
version: "3.9"
services:
  api:
    build: .
    ports:
      - "8080:8080"
    environment:
      - ConnectionStrings__DefaultConnection=Server=db;Database=ShopDb;User
          Id=sa;Password=YourPassword123!;TrustServerCertificate=True;
    depends_on:
      - db

  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      - ACCEPT_EULA=Y
      - MSSQL_SA_PASSWORD=YourPassword123!
    ports:
      - "1433:1433"
    volumes:
      - sql_data:/var/opt/mssql

volumes:
  sql_data:
```

One command, `docker compose up`, now starts both your API and its database, correctly networked together (the API reaches the database at the hostname `db`, not `localhost`). This is the standard way to spin up a full local dev environment for anyone who clones the repo.

4.2 CI/CD introduction

CI/CD automates what you'd otherwise do by hand every time you push code: run the tests, build the app, and (for CD) deploy it. **Continuous Integration** catches breakages early by running on every push. **Continuous Deployment** ships a passing build automatically, removing manual, error-prone deploy steps.

●●● .github/workflows/ci.yml — GitHub Actions pipeline yaml

```yaml
name: CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '8.0.x'

      - name: Restore dependencies
        run: dotnet restore

      - name: Build
        run: dotnet build --no-restore --configuration Release

      - name: Run tests
        run: dotnet test --no-build --configuration Release --verbosity normal

      - name: Build & push Docker image
        if: github.ref == 'refs/heads/main'
        run: |
          docker build -t ghcr.io/yourorg/shopapi:${{ github.sha }} .
          echo ${{ secrets.GHCR_TOKEN }} | docker login ghcr.io -u yourorg
              --password-stdin
          docker push ghcr.io/yourorg/shopapi:${{ github.sha }}
```

Read this as a checklist that runs automatically on every push: pull the code, install the .NET SDK, restore NuGet packages, build, run the test suite, and — only on the main branch, only if every prior step succeeded — build and publish a container image. A pull request that breaks the build or fails a test gets flagged before a human ever has to notice manually.

> **PRO TIP — Start with CI alone before automating deployment**
> Get comfortable with an automated build + test pipeline first. Add automatic deployment (true CD) once you trust your test suite to actually catch regressions — shipping broken code automatically is worse than not automating deployment at all.

4.3 Azure fundamentals for cloud deployment

Azure is Microsoft's cloud platform and the most common deployment target for .NET apps (unsurprising, since both are made by Microsoft, and the tooling integration is deep). You do not need to learn all of Azure — these four services cover the vast majority of a typical backend deployment.

Azure service	What it's for	Analogy
App Service	Fully-managed hosting for your web app/API — you push code or a container, Azure handles the VM, scaling, and OS patching	Like Heroku, but for Azure
Azure SQL Database	Fully-managed SQL Server in the cloud	SQL Server, without you managing the server
Azure Container Registry (ACR)	Private storage for your Docker images	Your own private Docker Hub
Azure Key Vault	Securely stores secrets, connection strings, and certificates	A managed, audited secrets manager

●●● **Deploying a container to Azure App Service via the Azure CLI** bash

```
# Log in and create a resource group (a logical container for related resources)
az login
az group create --name ShopApiRG --location eastus

# Create an App Service plan (defines the compute tier) and a Web App for
    containers
az appservice plan create --name ShopApiPlan --resource-group ShopApiRG
    --is-linux --sku B1
az webapp create --resource-group ShopApiRG --plan ShopApiPlan \
  --name shopapi-prod --deployment-container-image-name
     ghcr.io/yourorg/shopapi:latest

# Push app settings (env vars) — same pattern as the Docker section above
az webapp config appsettings set --resource-group ShopApiRG --name shopapi-prod
    \
  --settings ConnectionStrings__DefaultConnection="<your Azure SQL connection
     string>"
```

NOTE — You don't need to memorize CLI commands

In real teams, this deployment step almost always lives inside the CI/CD pipeline from Section 4.2 (adding an `azure/webapps-deploy` step) or is managed with Infrastructure-as-Code tools like Bicep or Terraform. What matters is understanding *what* each piece does — App Service hosts it, Azure SQL stores its data, Key Vault holds its secrets — so you can read and reason about a deployment pipeline someone else wrote.

KEY TAKEAWAYS

- Multi-stage Dockerfiles keep production images small by discarding the SDK/compiler after publishing.
- docker-compose spins up your API and its database together with one command — the standard local dev setup.
- CI runs your build + tests automatically on every push; CD extends that to automatic deployment once you trust the pipeline.
- App Service + Azure SQL + Key Vault covers most real-world .NET deployments without needing deep Azure expertise.

PRACTICE LAB — CHAPTER 4 EXERCISES

1. Write a multi-stage Dockerfile for your TaskApi from earlier chapters and successfully run it in a container, reachable at localhost:8080.

Hint / Goal: Confirm with docker ps that the container is running, then hit it with Postman.

2. Write a docker-compose.yml that runs your API alongside a SQL Server container, networked together.

Hint / Goal: The API's connection string should point at the service name (e.g. db), not localhost.

3. Set up a GitHub Actions workflow that restores, builds, and runs dotnet test on every push to main.

Hint / Goal: Push a deliberately failing test once to confirm the workflow correctly reports red.

4. Create a free-tier Azure account and deploy any 'Hello World' ASP.NET Core API to an App Service instance.

Hint / Goal: Note down what az webapp create actually provisions before you deploy anything more complex.

CHAPTER 5

Capstone — Production E-Commerce Backend

Every concept from Chapters 1-4 comes together here. You are going to design and build the backend for a real e-commerce platform — products, orders, authentication, an admin dashboard — structured the way it would actually be built on a real team. Treat this chapter as a spec to implement, not just something to read.

5.1 System design and project structure

Before writing a line of code, define the domain. An e-commerce backend needs, at minimum: Users (with roles), Products, Categories, Orders, OrderItems, and a Cart. Here's the entity relationship shape you're building toward:

●●● Core domain entities and their relationships — text

```
User (1) ----- (many) Order
Order (1) ----- (many) OrderItem ----- (many) Product
Category (1) ----- (many) Product
User (1) ----- (1) Cart ----- (many) CartItem ----- (many) Product
```

●●● Solution structure (Clean Architecture, applied for real) — text

```
ECommerce.sln
 |-- ECommerce.Domain/
 |     |-- Entities/  (User, Product, Category, Order, OrderItem, Cart,
     CartItem)
 |     |-- Enums/     (OrderStatus, UserRole)
 |-- ECommerce.Application/
 |     |-- DTOs/          (ProductDto, OrderDto, CreateOrderRequest ...)
 |     |-- Interfaces/    (IProductRepository, IOrderService, ITokenService ...)
 |     |-- Services/      (OrderService, ProductService ...)
 |-- ECommerce.Infrastructure/
 |     |-- Data/AppDbContext.cs
 |     |-- Repositories/  (ProductRepository, OrderRepository ...)
 |     |-- Migrations/
 |-- ECommerce.Api/
       |-- Controllers/   (AuthController, ProductsController,
           OrdersController, AdminController)
       |-- Middleware/    (ExceptionHandlingMiddleware)
       |-- Program.cs
```

Domain/Entities/Order.cs C#

```
namespace ECommerce.Domain.Entities
{
    public enum OrderStatus { Pending, Paid, Shipped, Delivered, Cancelled }

    public class Order
    {
        public int Id { get; set; }
        public int UserId { get; set; }
        public User User { get; set; } = null!;
        public OrderStatus Status { get; set; } = OrderStatus.Pending;
        public decimal TotalAmount { get; set; }
        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        public List<OrderItem> Items { get; set; } = new();
    }

    public class OrderItem
    {
        public int Id { get; set; }
        public int OrderId { get; set; }
        public Order Order { get; set; } = null!;
        public int ProductId { get; set; }
        public Product Product { get; set; } = null!;
        public int Quantity { get; set; }
        public decimal UnitPriceAtPurchase { get; set; }  // snapshot, not a
            live reference
    }
}
```

PRO TIP — Snapshot the price at time of purchase

Notice `UnitPriceAtPurchase` on `OrderItem`, rather than always reading `Product.Price` live. This is a real production pattern: if the product price changes next week, past orders must still show what the customer actually paid. Copying the value at the moment of purchase, instead of relying on a live foreign-key lookup, is the fix.

5.2 Authentication system integration

Wire together everything from Section 2.2, plus one addition every production auth system needs: **refresh tokens**. A short-lived JWT (e.g. 15 minutes) limits the damage if it leaks; a longer-lived refresh token lets the client silently get a new access token without forcing the user to log in again.

Controllers/AuthController.cs — register endpoint C#

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AppDbContext _context;
    private readonly TokenService _tokenService;

    public AuthController(AppDbContext context, TokenService tokenService)
    {
        _context = context;
        _tokenService = tokenService;
    }

    [HttpPost("register")]
    public async Task<IActionResult> Register(RegisterRequest request)
    {
        if (await _context.Users.AnyAsync(u => u.Email == request.Email))
            return Conflict("An account with this email already exists.");

        var user = new User
        {
            Email = request.Email,
            PasswordHash = BCrypt.Net.BCrypt.HashPassword(request.Password),
            Role = "Customer"
        };
        _context.Users.Add(user);
        await _context.SaveChangesAsync();
        return CreatedAtAction(nameof(Register), new { id = user.Id }, new {
            user.Id, user.Email });
    }
}
```

Controllers/AuthController.cs — login and refresh (same class, continued) C#

```
[HttpPost("login")]
public async Task<IActionResult> Login(LoginRequest request)
{
    var user = await _context.Users.FirstOrDefaultAsync(u => u.Email ==
        request.Email);
    if (user is null || !BCrypt.Net.BCrypt.Verify(request.Password,
        user.PasswordHash))
        return Unauthorized("Invalid email or password.");

    var accessToken = _tokenService.CreateAccessToken(user);
    var refreshToken = _tokenService.CreateRefreshToken();

    user.RefreshToken = refreshToken;
    user.RefreshTokenExpiry = DateTime.UtcNow.AddDays(7);
    await _context.SaveChangesAsync();

    return Ok(new { accessToken, refreshToken });
}

[HttpPost("refresh")]
public async Task<IActionResult> Refresh(RefreshRequest request)
{
    var user = await _context.Users.FirstOrDefaultAsync(u => u.RefreshToken
        == request.RefreshToken);
    if (user is null || user.RefreshTokenExpiry < DateTime.UtcNow)
        return Unauthorized("Refresh token invalid or expired.");

    var newAccessToken = _tokenService.CreateAccessToken(user);
    return Ok(new { accessToken = newAccessToken });
}
```

5.3 Admin dashboard API development

An admin dashboard is just a set of endpoints locked down to the Admin role, typically covering management operations (create/update products, view all orders, change order status) and simple aggregate reporting.

Controllers/AdminController.cs C#

```
[ApiController]
[Route("api/admin")]
[Authorize(Roles = "Admin")]     // applies to every action in this controller
public class AdminController : ControllerBase
{
    private readonly AppDbContext _context;
    public AdminController(AppDbContext context) => _context = context;

    [HttpGet("dashboard-summary")]
    public async Task<IActionResult> GetSummary()
    {
        var summary = new
        {
            TotalOrders = await _context.Orders.CountAsync(),
            TotalRevenue = await _context.Orders
                .Where(o => o.Status != OrderStatus.Cancelled)
                .SumAsync(o => o.TotalAmount),
            PendingOrders = await _context.Orders.CountAsync(o => o.Status ==
                OrderStatus.Pending),
            TotalCustomers = await _context.Users.CountAsync(u => u.Role ==
                "Customer")
        };
        return Ok(summary);
    }

    [HttpPut("orders/{id:int}/status")]
    public async Task<IActionResult> UpdateOrderStatus(int id, [FromBody]
        OrderStatus newStatus)
    {
        var order = await _context.Orders.FindAsync(id);
        if (order is null) return NotFound();

        order.Status = newStatus;
        await _context.SaveChangesAsync();
        return NoContent();
    }

    [HttpGet("products/low-stock")]
    public async Task<IActionResult> GetLowStock([FromQuery] int threshold = 5)
    {
        var products = await _context.Products
            .Where(p => p.StockQuantity <= threshold)
            .AsNoTracking()
            .ToListAsync();
        return Ok(products);
    }
}
```

Locking down the entire controller with a class-level `[Authorize(Roles = "Admin")]` attribute, rather than repeating it on every action, is the cleaner pattern once every endpoint in a controller shares the same access rule.

5.4 Production-ready architecture checklist

Before calling any backend “production-ready,” run it against this checklist — it consolidates everything from Chapters 1 through 5 into the questions a senior engineer or a code review would actually ask.

Area	Checklist item
Architecture	Business logic separated from controllers and EF Core (Clean Architecture layers)
Data	Migrations committed to source control; no manual schema changes in production
Auth	Passwords hashed with BCrypt; short-lived JWT + refresh token flow; role/policy checks on every sensitive endpoint
Error handling	Global exception middleware; no stack traces returned to clients
Logging	Structured logs with request context; errors logged with enough detail to debug without reproducing locally
Performance	AsNoTracking() on read-only queries; Include() used deliberately, no N+1 queries
Security	HTTPS enforced; CORS allow-list; rate limiting on auth endpoints; secrets outside source control
Deployment	Containerized with a multi-stage Dockerfile; CI runs tests on every push
Documentation	Swagger/OpenAPI enabled for every endpoint; a README describing setup and environment variables

KEY TAKEAWAYS

- A real e-commerce backend is the sum of every prior chapter: layered architecture, EF Core relationships, JWT auth with refresh tokens, role-locked admin endpoints, and a deployable container.
- Snapshot values that must not change retroactively (like an order's price) instead of relying on a live foreign-key lookup.
- Lock down an entire controller with a class-level [Authorize] attribute when every action shares the same access rule.
- The production-readiness checklist above is worth revisiting before every real deployment, not just this capstone.

PRACTICE LAB — CHAPTER 5 EXERCISES

1. Implement the full domain model (User, Product, Category, Order, OrderItem, Cart, CartItem) and generate your first migration for the whole schema.

Hint / Goal: Confirm every foreign key relationship shows up correctly in your database's schema diagram.

2. Build the complete AuthController shown above, including the refresh token flow, and test the full cycle in Postman: register -> login -> call a protected endpoint -> refresh -> call again with the new token.

Hint / Goal: Store the refresh token as an HttpOnly cookie in a real frontend integration; a raw JSON body is fine for this exercise.

3. Build a POST /api/orders endpoint that accepts a list of {productId, quantity}, checks stock, computes the total server-side (never trust a client-submitted price), creates the Order + OrderItems, and decrements stock.

Hint / Goal: Wrap the stock check and the order creation in a single transaction so a failure partway through doesn't leave inconsistent data.

4. Build the AdminController's dashboard-summary and low-stock endpoints, and verify a Customer-role JWT gets a 403 on both.

Hint / Goal: This confirms your role-based authorization is actually enforced, not just present in code.

5. Run your finished project through the Production-Ready Architecture Checklist above, item by item, and fix anything marked incomplete.

Hint / Goal: Treat every unchecked box as a real bug, not a nice-to-have.

6. Capstone extension (optional, resume-worthy): containerize the finished API, wire up a GitHub Actions pipeline that runs your tests on every push, and deploy it to a free-tier Azure App Service instance.

Hint / Goal: This single exercise, completed end-to-end, is a legitimate portfolio project.

APPENDIX

Cheat Sheets & Interview Quick-Reference

A condensed reference for revision the night before an interview or a project kickoff — not a replacement for the chapters, but a fast way back to the key facts.

A.1 Service lifetimes, one line each

Lifetime	Rule of thumb
Transient	Cheap, stateless, no shared state — new instance every time
Scoped	Default choice for business services and DbContext — one per HTTP request
Singleton	App-wide shared state/config — created once, be careful what it depends on

A.2 HTTP status codes you'll use constantly

Code	Meaning	Typical trigger
200 OK	Success	Successful GET, PUT, PATCH
201 Created	Resource created	Successful POST — pair with CreatedAtAction
204 No Content	Success, nothing to return	Successful PUT/DELETE
400 Bad Request	Client sent invalid data	Failed model validation
401 Unauthorized	Caller is not authenticated	Missing/invalid JWT
403 Forbidden	Caller is authenticated but not allowed	Wrong role/policy
404 Not Found	Resource doesn't exist	Lookup by id returned null
409 Conflict	Request conflicts with current state	Duplicate email on register
429 Too Many Requests	Rate limit exceeded	Rate limiter policy triggered
500 Internal Server Error	Unhandled exception	Caught by global exception middleware

A.3 Common interview questions this book prepares you for

- What's the difference between AddScoped, AddTransient, and AddSingleton, and what breaks if you get it wrong?
- Walk me through what happens, middleware by middleware, when a request with a JWT hits a protected endpoint.
- What's the N+1 query problem, and how does EF Core's Include() solve it?
- When would you choose microservices over a monolith, and what does that choice actually cost you?
- How would you design the database schema for an e-commerce order, and why snapshot the price?
- What's the difference between authentication and authorization, and where does each happen in the pipeline?
- Why does a multi-stage Dockerfile produce a smaller, safer production image?
- What's the difference between Code-First and Database-First, and when would you use each?

A.4 Command-line reference

Every CLI command used across this book, in one place bash

```
# Project setup
dotnet new webapi -n ProjectName
dotnet new classlib -n ProjectName.Domain
dotnet run
dotnet watch run                      # auto-restart on file changes

# Packages
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package BCrypt.Net-Next
dotnet add package Serilog.AspNetCore

# EF Core
dotnet tool install --global dotnet-ef
dotnet ef migrations add MigrationName
dotnet ef database update
dotnet ef migrations script
dotnet ef dbcontext scaffold "<connection-string>"
    Microsoft.EntityFrameworkCore.SqlServer -o Models

# Docker
docker build -t appname:latest .
docker run -d -p 8080:8080 appname:latest
docker compose up
docker compose down

# Azure CLI
az login
az group create --name RG --location eastus
az webapp create --resource-group RG --plan PlanName --name app-name
```

PRO TIP — What to do next

Rebuild the Chapter 5 capstone from scratch, without looking at the code samples, using only the section headings as prompts. If you can do that in a weekend, you are genuinely ready for production ASP.NET Core work.